



机器学习

第9章 遗传算法

Genetic Algorithms



主要内容

- 基本概念：
 - 适应度，交叉，变异，拥挤效应
- 遗传算法框架
- 如何设计遗传算法
 - 编码，参数选择，适应度函数设计
- 算法实例

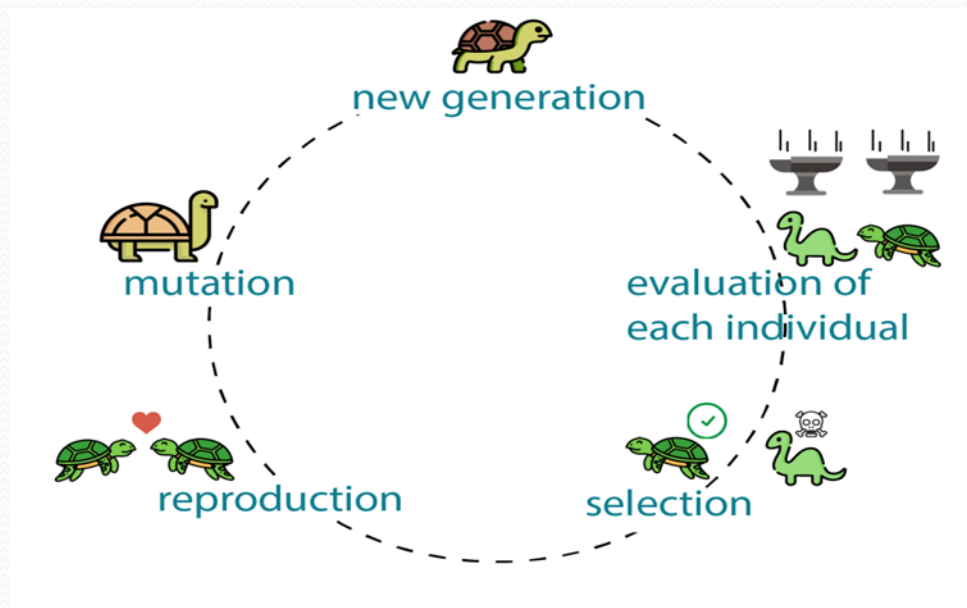


9.1 动机

- 遗传算法是一种受生物进化启发的学习方法
 - GAs通过不断变异和重组当前已知的最好假设来生成后续的假设
- GA发展得益于：
 - 进化被认为是健壮的
 - GA搜索的假设空间中，假设的各个部分相互作用，每一部分对总的假设适应度的影响难以建模
 - 易于并行化

9.2 遗传算法

- 假设适应度
 - 适应度：评估假设优劣
 - 最好的假设拥有最高的适应度
- 遗传算法的结构：
 - 迭代更新一个群体
 - 评估群体中的个体
 - 生成新群体

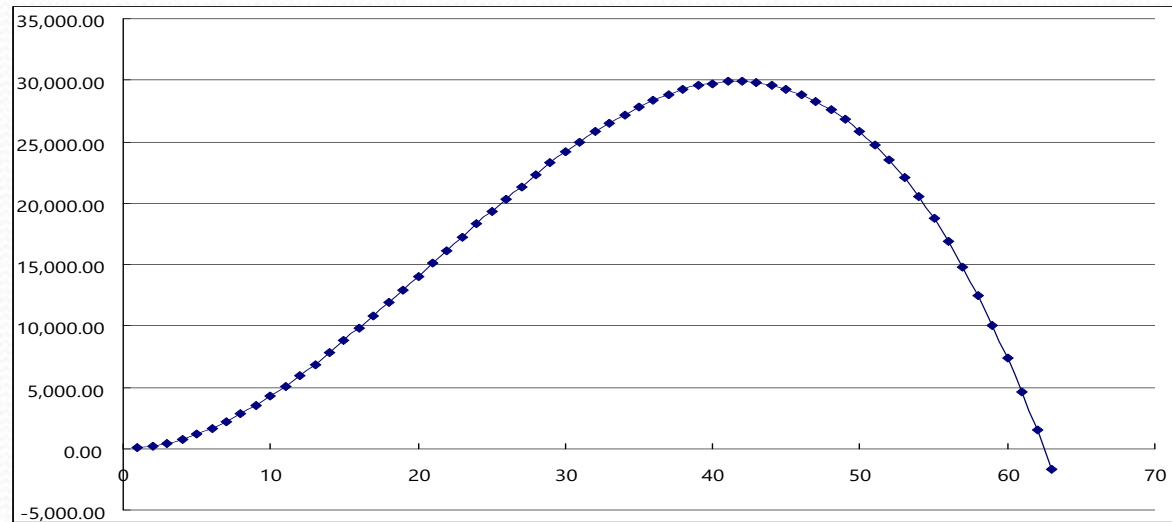


Let's start from a problem

$$\text{Max } F(X) = -0.83X^3 + 52X^2 - 8.5X + 8$$

$$\text{Subject to: } 0 \leq X \leq 61$$

$$X \in \text{Integer}$$



■ 解决方案?

- 理论法，如果没有解？
- 直接法 (牛顿,共轭梯度等)
- 蛮力法，如果搜索空间很大？
- 其他方法？

“A Dumb Solution”

“盲目生成测试” 算法：

重复

 生成一个任意可能的解

 测试解的性能

直到解足够好

我们能使用这个方法吗？

- 有时-可以:
 - 如果解空间较小
 - 你有足够时间
- 大多数时候-不可以:
 - 可能的解空间太大
 - 没时间去一一尝试

一个“less-dumb”的方法 (GA)

生成一个任意解的集合

重复

- 测试集合中每个解（将它们排序）

- 从集合中移除一些不好的解

- 复制一些好的解

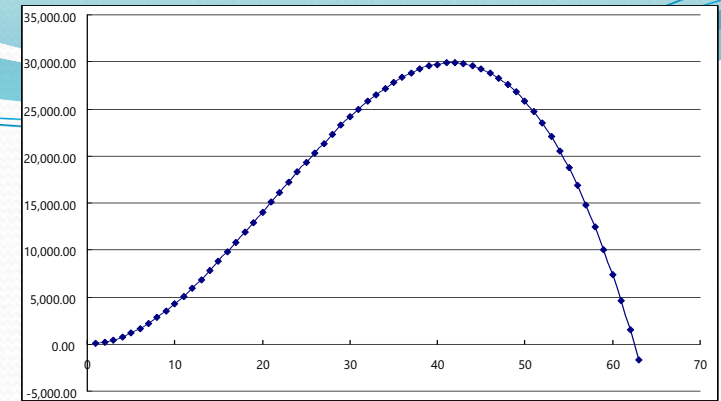
- 对它们做出一些小的改变

直到最好的解足够好

$$\text{Max } F(X) = -0.83X^3 + 52X^2 - 8.5X + 8$$

$$\text{Subject to: } 0 \leq X \leq 61$$

$$X \in \text{Integer}$$



Step 1: 编码

		X		$F(X)$						
<i>pop 1</i>	<table border="1"><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td></tr></table>	1	0	1	0	1	1	43		29799.69
1	0	1	0	1	1					
<i>pop 2</i>	<table border="1"><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	0	0	0	1	1	0	6		1649.72
0	0	0	1	1	0					
<i>pop 3</i>	<table border="1"><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	1	0	0	0	1	1	35		27824.25
1	0	0	0	1	1					

Step 2: 生成群体

pop 1	0	1	1	1	0	1	29
pop 2	1	0	1	0	1	0	42
pop 3	0	1	1	1	1	1	31
pop 4	0	1	0	1	0	1	21
pop 5	1	0	0	1	1	1	39
...	.	.	.	.	.	.	.
...	.	.	.	.	.	.	.
pop 19	0	1	0	0	0	1	17
pop 20	0	0	1	1	1	1	15

	<i>X</i>							<i>F(X)</i>	<i>survival rate</i>
Pop 1	0	1	1	1	0	1	29	23250.63	5.77%
Pop 2	1	0	1	0	1	0	42	29885.96	7.41%
Pop 3	0	1	1	1	1	1	31	24989.97	6.20%
Pop 4	0	1	0	1	0	1	21	15074.87	3.74%
Pop 5	1	0	0	1	1	1	39	29533.73	7.33%
Pop 6	1	0	1	1	1	1	47	28303.41	7.02%
Pop 7	0	1	0	1	0	1	21	15074.87	3.74%
Pop 8	0	1	0	0	0	1	17	10813.71	2.68%
Pop 9	0	0	1	1	1	1	15	8779.25	2.18%
Pop 10	1	1	0	1	1	1	55	18749.25	4.65%
Pop 11	0	1	1	1	0	1	29	23250.63	5.77%
Pop 12	0	1	0	1	1	1	23	17221.89	4.27%
Pop 13	1	0	1	0	1	1	43	29799.69	7.39%
Pop 14	0	1	1	0	1	0	26	20350.92	5.05%
Pop 15	1	0	0	0	1	1	35	27824.25	6.90%
Pop 16	0	0	1	1	1	1	15	8779.25	2.18%
Pop 17	0	1	0	1	1	1	23	17221.89	4.27%
Pop 18	0	0	0	1	0	1	5	1161.75	0.29%
Pop 19	1	0	1	0	1	1	43	29799.69	7.39%
Pop 20	0	1	1	1	0	1	29	23250.63	5.77%

$$\text{Survival Rate} = \frac{\text{fitness}}{\text{sum}(\text{fitness})}$$

Step 3: 交叉

pop 1	0	1	1	1	0	1	29
pop 2	1	0	1	0	1	0	42
pop 3	0	1	1	1	1	1	31
pop 4	0	1	0	1	0	1	21
pop 5	1	0	0	1	1	1	39
...	.	.	.	.	.	.	.
...	.	.	.	.	.	.	.
pop 19	0	1	0	0	0	1	17
pop 20	0	0	0	1	1	0	6

*Randomly
select
parents*

Parent 1

0	1	1	1	0	1
---	---	---	---	---	---

Parent 2

1	0	0	1	1	1
---	---	---	---	---	---

Crossover point



Offspring 1

0	1	1	1	1	1
---	---	---	---	---	---

X F(X)

31 24989.97

Offspring 2

1	0	0	1	0	1
---	---	---	---	---	---

37 28839.51

Step 4: 变异

		X	$F(X)$			X	$F(X)$												
Offspring 1	<table><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	0	1	1	1	1	1	31	24989.97										
0	1	1	1	1	1														
Offspring 2	<table><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td></tr></table>	1	0	0	1	0	1	37	28839.51	→		<table><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr></table>	1	0	0	1	1	1	39 29533.73
1	0	0	1	0	1														
1	0	0	1	1	1														
Offspring 3	<table><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td></tr></table>	0	0	1	0	1	0	10	4293.00										
0	0	1	0	1	0														
.....																			
Offspring 20	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	0	0	0	1	1	0	6	1649.72										
0	0	0	1	1	0														

↑
随机选取变异位点

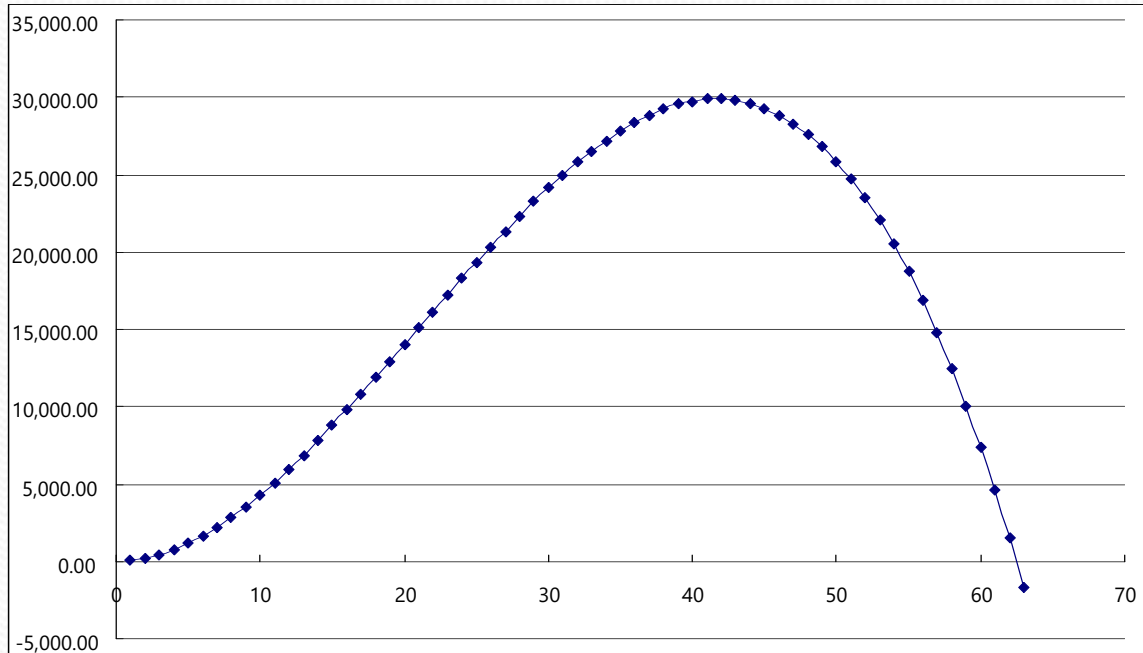
Step 5: 回到步骤2

When converged!

	X						$F(X)$	
Pop 1	1	0	1	0	1	0	42	29885.96
Pop 2	1	0	1	0	1	0	42	29885.96
Pop 3	1	0	1	0	1	0	42	29885.96
Pop 4	1	0	1	0	1	0	42	29885.96
Pop 5	1	0	1	0	1	0	42	29885.96
Pop 6	1	0	1	0	1	0	42	29885.96
Pop 7	1	0	1	0	1	0	42	29885.96
Pop 8	1	0	1	0	1	0	42	29885.96
Pop 9	1	0	1	0	1	0	42	29885.96
Pop 10	1	0	1	0	1	0	42	29885.96
Pop 11	1	0	1	0	1	0	42	29885.96
Pop 12	1	0	1	0	1	0	42	29885.96
Pop 13	1	0	1	0	1	0	42	29885.96
Pop 14	1	0	1	0	1	0	42	29885.96
Pop 15	1	0	1	0	1	0	42	29885.96
Pop 16	1	0	1	0	1	0	42	29885.96
Pop 17	1	0	1	0	1	0	42	29885.96
Pop 18	1	0	1	0	1	0	42	29885.96
Pop 19	1	0	1	0	1	0	42	29885.96
Pop 20	1	0	1	0	1	0	42	29885.96

$$F(X) = 0.83X^3 + 52X^2 - 8.5X + 8$$

Subject to: $0 \leq X \leq 61$
 $X \in \text{Integer}$





遗传算法的原型

- GA(Fitness, fitness_threshold, p,r,m)
Fitness: 适应度评分函数;
fitness_threshold: 指定终止判据的阈值
p:群体中包含的假设数量
r:每一步中通过交叉取代群体成员的比例;
m:变异率



initialize population : P

Evaluate : For each h in P, compute Fitness(h)

while[max Fitness(h)] < Fitness_threshold do

{

create a new generation, P_s :

1. **Select**: select $(1-r)p$ members of P to add to P_s .

$$\text{Pr}(h_i) = \text{Fitness}(h_i) / \sum \text{Fitness}(h_j)$$

2. **Crossover**: select $r*p/2$ pairs of hypotheses from P.

applying the crossover operator. Add all offspring to P_s

3. **Mutate**: choose m percent of the members of P_s .

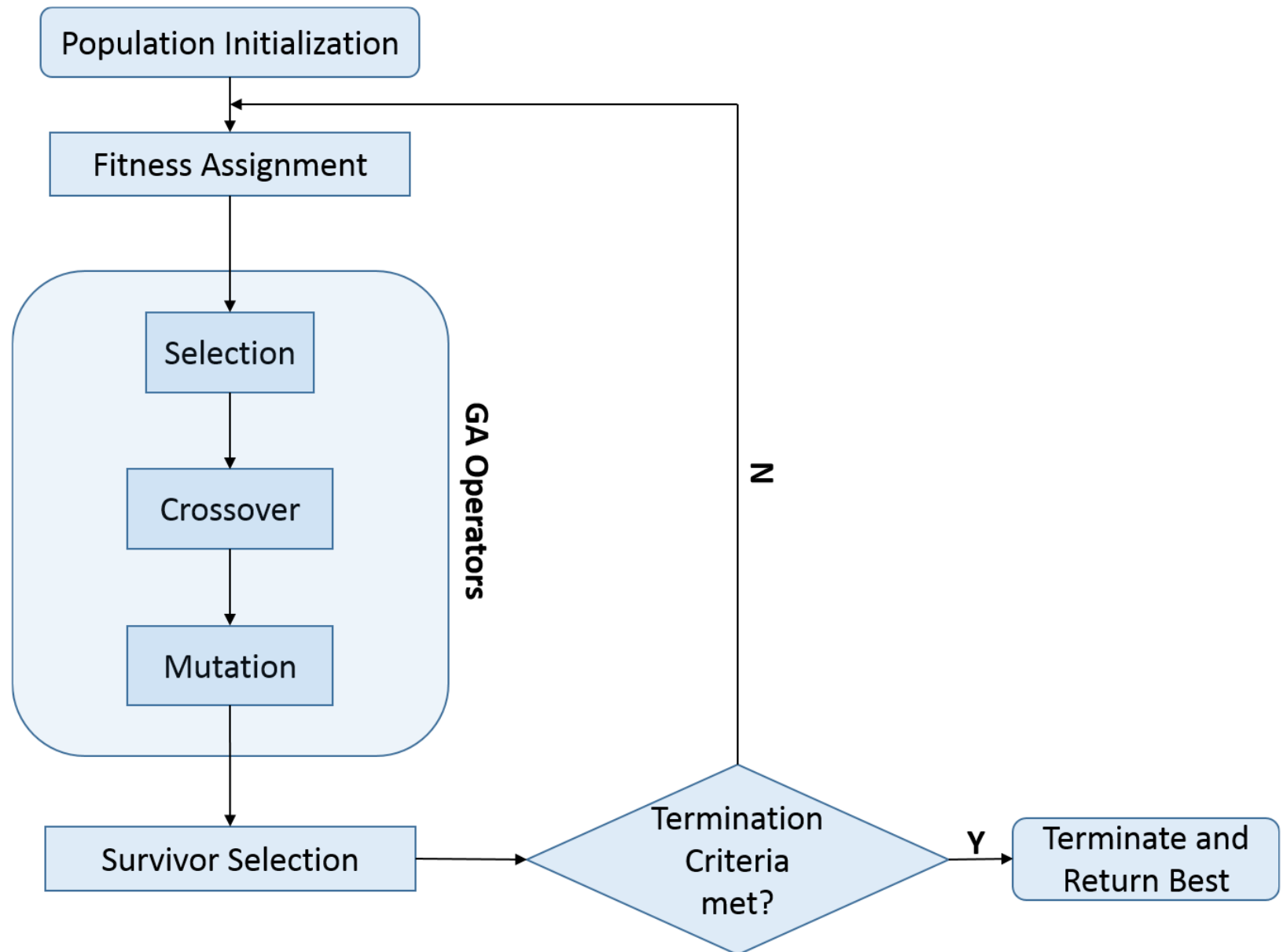
invert one randomly selected bit in its representation.

4. update: $P = P_s$

5. Evaluate: for each h in P, compute Fitness(h)

}

Return the hypothesis from P that has the highest fitness



GA的参数

- GA有以下参数:
 - 交叉率Crossover rate
 - 变异率Mutation rate
 - 群体大小Population size
 - 编码Encoding
 - 交叉变异类型Crossover and mutation type
 - 选择Selection
- 没有一种通用理论能帮助你为各种问题调整参数
 - 你可能需要针对特定问题尝试自己的GA



9.2.1 表示假设

- 假设常用二进制位串表示
 - 便于用变异和交叉遗传算子来操作
- 例子: 编码if-then规则
 - If wind=strong, then playTennis =yes
 - 属性编码
 - **Wind**: Strong, weak **Outlook**: Sunny, Overcast, Rain
 - 每位对应一个属性值, 如: 长为2的位串: **10**
 - 属性不被规则的前件约束, “不关心属性值”
 - 例如: Outlook=111



9.2.1 表示假设

- 规则编码
 - 连接相应的位串
 - outlook wind PlayTennis
 - 111 10 10
- 规则集的表达：
 - 连接各个规则的位串
- 让句法合法的位串表示一个有意义的假设
 - 111 10 11?
 - 如何避免？
- 规则适应度
 - 分类训练样例正确率

9.2.2 遗传算子



- 最常见的算子: 交叉和变异
- 交叉算子:
 - 通过复制双亲的选定位, 从双亲的字符串中产生两个新的后代
 - 交叉掩码:
 - 单点交叉

$A: 10110111\textcolor{red}{00} \quad 1111111100 \quad A': 10110111\textcolor{green}{11}$
 $B: 00011100\textcolor{green}{11} \quad \longrightarrow \quad B': 00011100\textcolor{red}{00}$

- 两点交叉

$A: \textcolor{red}{101}10111\textcolor{red}{00} \quad 0001111100 \quad A': \textcolor{green}{000}10111\textcolor{green}{11}$
 $B: \textcolor{green}{000}11100\textcolor{green}{11} \quad \longrightarrow \quad B': \textcolor{red}{101}11100\textcolor{red}{00}$

- 均匀交叉

$A: \textcolor{red}{101}1\textcolor{red}{00}11\textcolor{red}{00} \quad 0001101100 \quad A': \textcolor{green}{000}10\textcolor{green}{11}11$
 $B: \textcolor{green}{000}11100\textcolor{green}{11} \quad \longrightarrow \quad B': \textcolor{red}{101}11\textcolor{red}{00}00\textcolor{red}{00}$



9.2.2 遗传算子 (2)

- 变异算子:
 - 对单亲的位串随机改变来生成后代
 - 变异经常在交叉之后进行的

Before Mutation

A5

1	1	1	0	0	0
---	---	---	---	---	---

After Mutation

A5

1	1	0	1	1	0
---	---	---	---	---	---



- 规则的变异、交叉算子设计技巧举例：

- 变异算子：选取合适位进行变异

- 交叉算子：两点交叉的扩展

- 选取 h_1 的交叉点，第1位，第8位

- $h_1=1 \{0 \quad 0 \quad 1 \quad 1 \quad 1\} 0 \quad 0 \quad (d_1=1, d_2=2)$

- h_2 交叉点需要有同样的 d_1, d_2

- $h_2=0 \{1 \quad 1\} 1 \quad 0 \quad 10 \quad 0 \quad 1 \quad 0 \quad (1,3) (1,8) (6,8)$

- $h_1=1 \{1 \quad 1\} 0 \quad 0$

- $h_2=0 \{0 \quad 0 \quad 1 \quad 1 \quad 1\} 1 \quad 0 \quad 10 \quad 0 \quad 1 \quad 0$

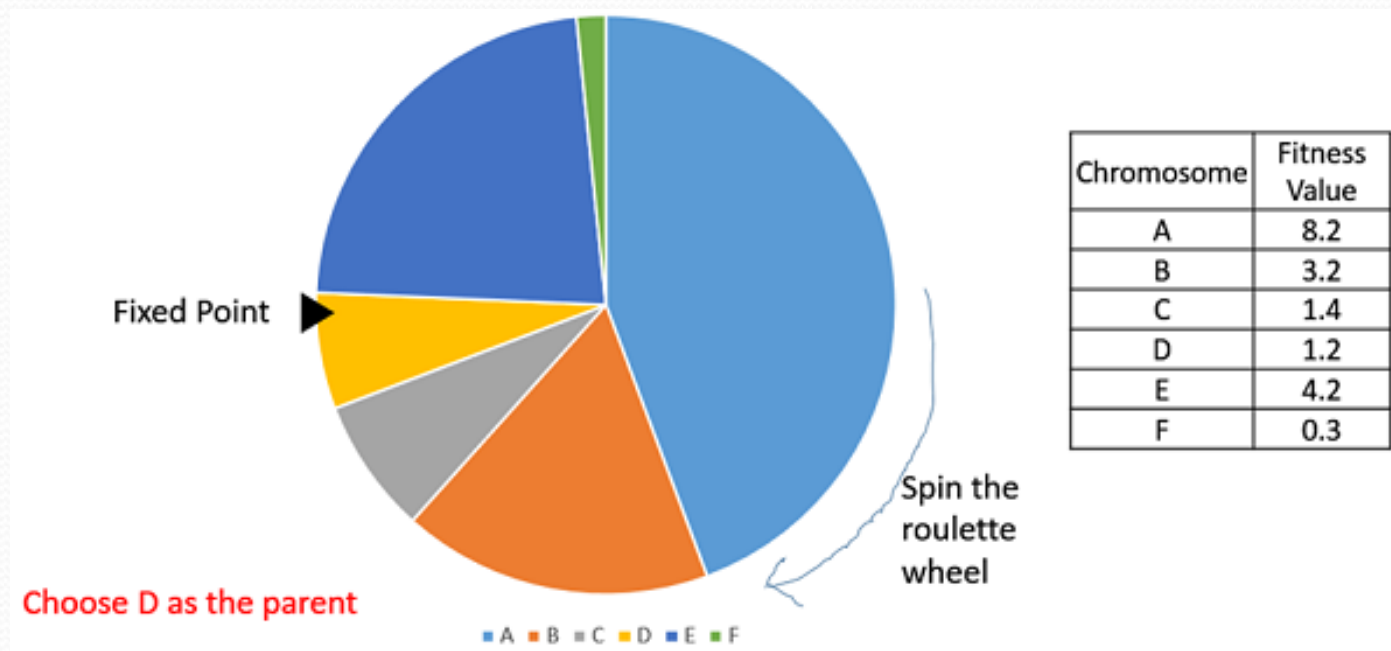
- 适应度函数：在训练数据上的分类准确率

- $\text{Fitness}(h) = (\text{correct}(h))^2$



9.2.3 适应度函数和假设选择

- 适应度函数定义了对假设进行排序的标准
- 选择假设的方法
 - 适应度比例选择(轮盘赌选择)



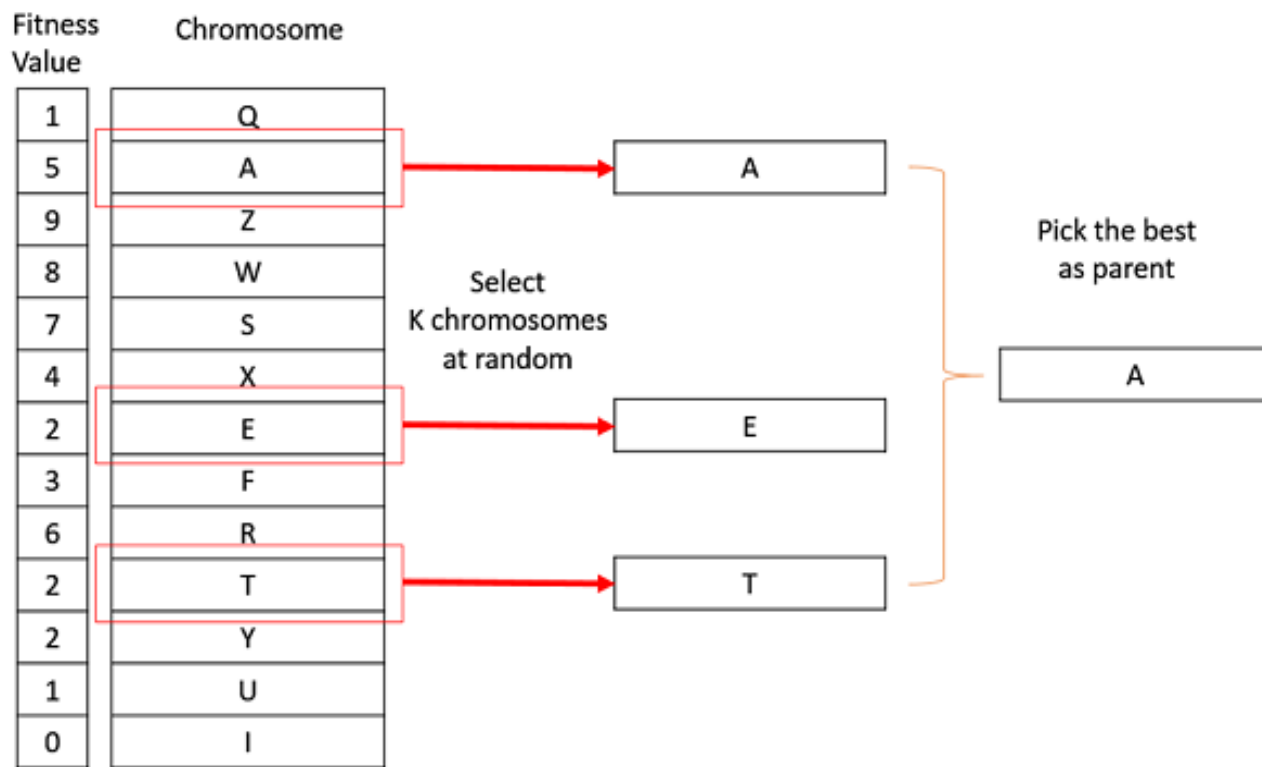


9.2.3 适应度函数和假设选择

● 选择假设的方法

● 锦标赛选择

- 随机选取两个(或多个)假设
- 依概率 p , 选择 $h_{\text{high}}(1-p)$; 依概率 $1-p$, 选择 h_{low}
- 更加多样化的群体





9.2.3 适应度函数和假设选择

- 选择假设的方法
 - 排序选择
 - 按适应度排序假设
 - 选中概率和假设的排序成比例，与适应度无关

Chromosome	Fitness Value	Rank
A	8.1	1
B	8.0	4
C	8.05	2
D	7.95	6
E	8.02	3
F	7.99	5

旅行商问题

∞ 这个问题已被证明为**NP**-完全问题

∞ 解决方案:

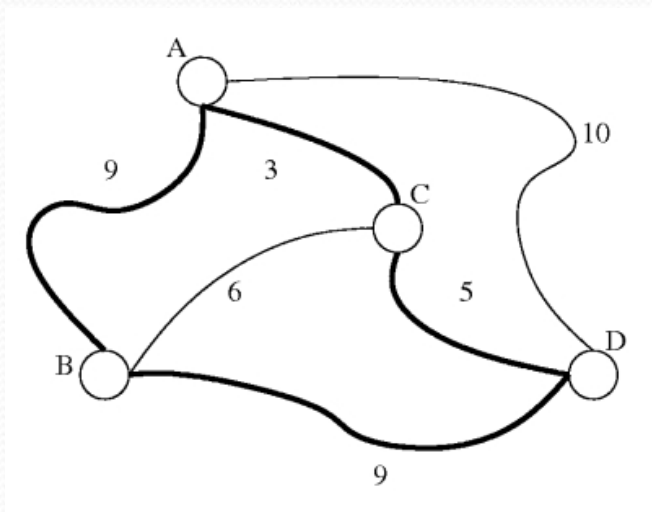
∞ **heuristics,**

∞ **cutting-plane methods,**

∞ **branch-and-cut**

∞ 如何用**GA**解决?

(1) 编码



	<i>1st city on tour</i>	<i>2nd city on tour</i>	<i>last city on tour</i>
City #			

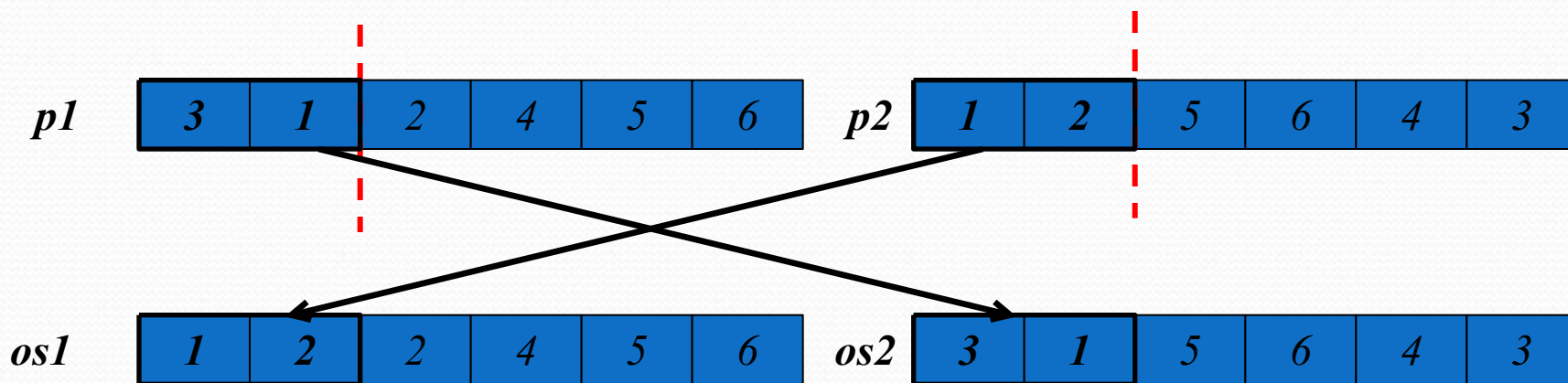
- 例如，我们有6个城市，一个旅游方案为3-1-2-4-5-6

<i>3</i>	<i>1</i>	<i>2</i>	<i>4</i>	<i>5</i>	<i>6</i>
----------	----------	----------	----------	----------	----------

(2) 适应度评估

- 给定一条染色体，只需计算总距离即可
- 距离越短，染色体的适应度越高

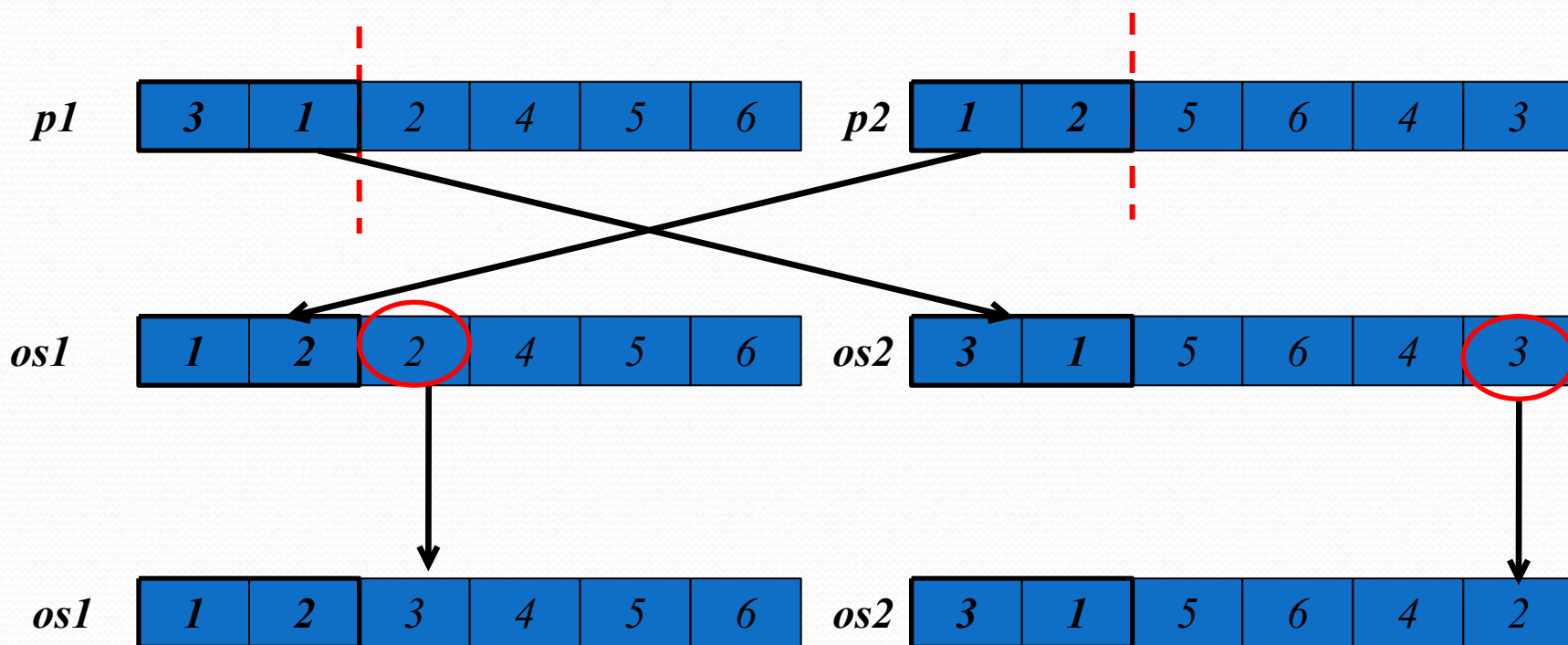
(3) 交叉



- 有什么问题吗？

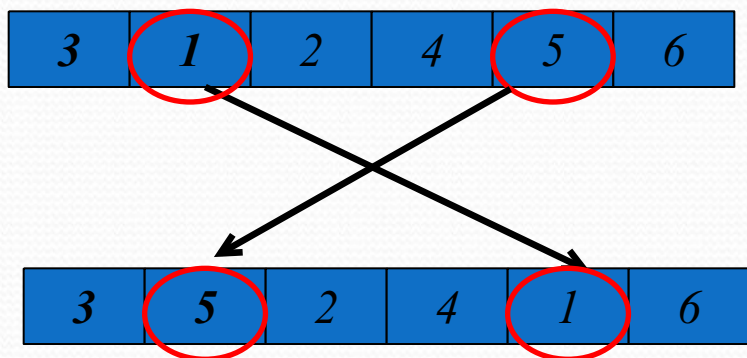
如何解决这个问题

- 修复算子

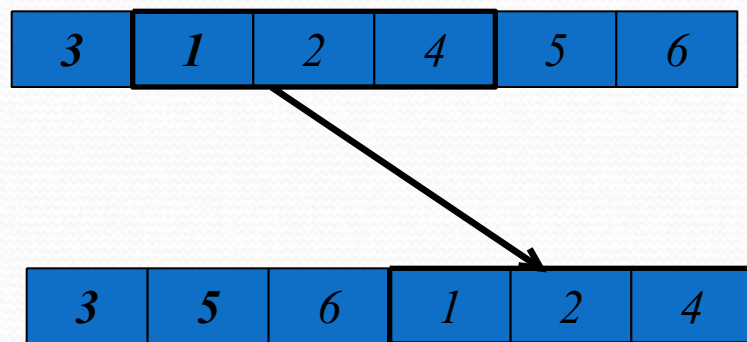


(3) 变异

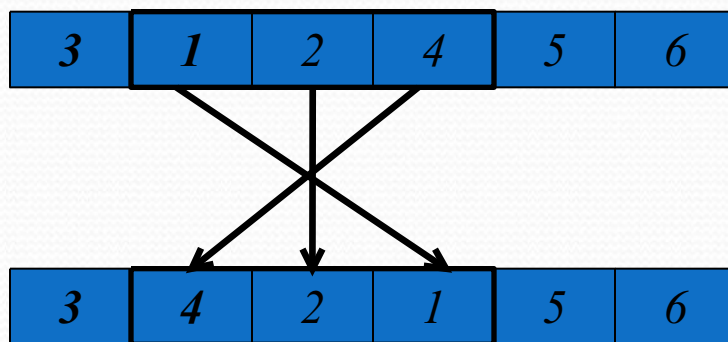
- 交叉变异



- 移位变异



- 简单倒置变异





9.4 假设空间搜索

- 遗传算法采用随机柱状搜索方法来寻找最大适应度假设
 - 梯度下降搜索：从一个 h 平滑地移动到新的 h
 - 遗传算法：移动可能非常突然，用完全不同假设替代双亲
 - 不太可能陷入局部极小值
- 实际困难：拥挤问题
 - 适应度高的个体迅速繁殖，因此该个体和它非常相似的个体占据了群体的很大一部分
 - 减少群体多样性，减缓进一步进化



假设空间搜索 (2)

- 降低拥挤的策略
 - 修改选择函数
 - 使用锦标赛选择或排序选择
 - 适应度共享
 - 根据群体中与某个体相似的个体数量，减小该个体的适应度
 - 对可重组生成后代的个体种类进行限制
 - 只允许最相似的个体重组
 - 按空间分布个体，并且仅允许相邻的个体重组

总结

- GA优点:

1. 算法不需要太多的先验知识或计算损失函数的梯度
2. 多目标函数优化
3. 可并行
4. 易处理庞大难理解的搜索空间
5. 广泛应用于优化问题求解

总结

GA缺点:

1. 假设的编码表示较难定义
2. 理想的适应度函数较难定义
3. 发生过早收敛
4. 选择诸如种群规模, 变异率, 交叉率, 适应度函数之类的各种参数的问题
5. 无法使用梯度
6. 终止条件判断较困难
7. 需要大量的适应度计算